Themes

Themes control how *Calepin* renders HTML pages and notebook outputs. A theme can provide MiniJinja HTML templates, shared or local partials, CSS, JavaScript, and a Typst notebook template for PDF, SVG, PNG, and HTML output.

Choosing a theme

The default theme is `calepin`. Select a different built-in or local theme with `theme` in a website's `calepin.toml`:

```
theme = "calepin"           # the default documentation theme
theme = "academic"          # a built-in essay/blog theme
theme = "themes/my-theme"   # a local theme directory
theme = false               # no theme: raw, unstyled output
```

The same values work with `--theme` on `calepin compile` and inside a document with `#calepin.setup(theme: ...)`:

```
calepin compile paper.typ --theme calepin
```

When several theme settings are present during a compile, the command line wins, then the document, then `calepin.toml`. `calepin watch` does not have a `--theme` option; it uses the document setting when present, otherwise the website's `calepin.toml` setting, otherwise the default theme.

Calepin ships with two built-in themes:

- **calepin**: the default documentation site layout, with sidebar navigation, a top bar, previous and next page links, a table of contents, dark mode, copy buttons on code blocks, and rendered/source/PDF view switching.
- **academic**: a reading-first essay and blog layout with a centered narrow text column, margin-note support, top navigation, dark mode, copy buttons on code blocks, and shared Calepin search and language controls.

Ejecting and local themes

Built-in themes are compiled into the *Calepin* binary, so you cannot edit them directly. Instead, copy one into your project and edit the copy:

```
calepin new theme                     # copies the default theme to themes/calepin/
calepin new theme --theme academic    # copies the academic theme to themes/academic/
calepin new theme themes/my-theme --theme academic
```

Then point your site or compile command at the copy:

```
theme = "themes/calepin"
```

The copy is yours: edit its HTML, CSS, JavaScript, `theme.toml`, and notebook template files freely, and check them into version control. *Calepin* upgrades will never touch them. Ejected shared files are written beside the theme in `themes/shared/`.

A local theme can be small. Missing entry files fall back to the built-in `calepin` theme, so a local theme can override just `layouts/webpage.html` or just `layouts/notebook.html` or `notebook.typ.jinja`. Supporting files such as partials, styles, and scripts come from the selected theme plus any imports declared in `theme.toml`.

Structure

A theme can provide templates for website pages, single-document HTML, and Typst-level notebook rendering:

```
themes/my-theme/
  theme.toml        # optional shared partial/CSS/JS imports
  layouts/
    webpage.html    # layout for website pages
    notebook.html   # layout for a single notebook rendered to HTML
    landing.html    # optional page-specific website layout
  partials/         # theme-local MiniJinja fragments
  styles/           # theme-local CSS files
  scripts/          # theme-local JavaScript files
  notebook.typ.jinja # optional Typst notebook template
themes/shared/      # optional local source for imported shared files
  partials/
  styles/
  scripts/
  typst/
```

`layouts/webpage.html`, `layouts/notebook.html`, page-specific files in `layouts/`, and files in `partials/` use the MiniJinja template language.

HTML templates

For notebooks compiled as single HTML documents, the relevant layout is `layouts/notebook.html`. For websites, most pages use `layouts/webpage.html`.

Templates can access:

- `doc.head`
- `doc.body_open`
- `doc.body`
- `doc.body_close`
- `doc.title`
- `site.sidebar`
- `site.sidebar_sections`
- `site.navbar_left`
- `site.navbar_center`
- `site.navbar_right`
- `site.languages`
- `site.translations`
- `site.language`
- `site.toc`
- `site.title`
- `site.description`
- `site.base_url`
- `site.logo`
- `site.logo_alt`
- `site.home_url`
- `site.favicon`
- `site.current_url`
- `site.page_title`
- `styles`

- `scripts`
- `syntax_css`
- `theme`
- `target`

Navigation entries expose `href`, `label`, `label_html`, and `active`.

Here is a minimal `layouts/notebook.html` for single-file HTML output:

```
{{ doc.head }}
  <title>{{ doc.title }}</title>
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/@picocss/pico@2/css/pico.
min.css">
  {% for style in styles %}
  <style>
{{ style.css }}
  </style>
  {% endfor %}
{{ doc.body_open }}
  <header class="document-header">
    <a href="index.html">Home</a>
    <button type="button" data-calepin-theme-toggle>Theme</button>
  </header>
  <main class="container">
    {{ doc.body }}
  </main>
  {% for script in scripts %}
  <script>
{{ script.content }}
  </script>
  {% endfor %}
{{ doc.body_close }}
```

`doc.head`, `doc.body_open`, and `doc.body_close` come from Typst's generated HTML. Keep them in the template unless you are deliberately replacing the whole document shell.

Website layouts

A website page can select a different HTML layout from the active theme with `layout` in its `<website-metadata>`:

```
#metadata((
  title: "Landing page",
  layout: "layouts/landing.html",
)) <website-metadata>
```

The `layout` value is an explicit path inside the active theme. *Calepin* uses it exactly as written: it does not add `layouts/`, does not add `.html`, and does not fall back to `layouts/webpage.html` if the file is missing. The path must name a relative `.html` file that stays inside the theme directory.

For example, with `theme = "themes/my-theme"`, the metadata above resolves to:

`themes/my-theme/layouts/landing.html`

Page-specific layouts receive the same MiniJinja context as `layouts/webpage.html`, including `doc`, `site`, `styles`, and `scripts`, and they share the active theme's partials, shared imports, styles, and scripts.

Partials

A partial is a reusable MiniJinja template fragment stored under `partials/`. Use partials for repeated HTML such as a header, footer, navigation list, search box, or analytics snippet.

Include a partial from `layouts/webpage.html`, `layouts/notebook.html`, a page-specific layout, or another partial:

```
{% include "partials/header.html" %}
```

Partials receive the same template context as the file that includes them, so a partial included by `layouts/webpage.html` can read `site.title`, `site.navbar_left`, `styles`, `scripts`, and the other website template variables.

For example, `partials/site-header.html` can render the brand and top navigation:

```
<header class="site-header">
  <nav>
    <ul>
      <li>
        <a href="{{ site.home_url }}">
          {% if site.logo %}
          <img src="{{ site.logo }}" alt="{{ site.logo_alt }}">
          {% elif site.title %}
          {{ site.title }}
          {% else %}
          Home
          {% endif %}
        </a>
      </li>
      {% for item in site.navbar_left %}
      {% include "partials/nav-item.html" %}
      {% endfor %}
    </ul>
    <ul>
      {% for item in site.navbar_right %}
      {% include "partials/nav-item.html" %}
      {% endfor %}
    </ul>
  </nav>
</header>
```

Then `partials/nav-item.html` can render one navigation link:

```
<li>
  <a href="{{ item.href }}" aria-label="{{ item.label }}"{% if item.active %} aria-
current="page"{% endif %}>
    {{ item.label_html }}
  </a>
</li>
```

Shared files

Themes can opt into shared partials, CSS, and JavaScript with `theme.toml`. These are the pieces that the built-in themes use for metadata, stylesheet/script wiring, typography, syntax highlighting, code output, dark mode, language selection, theme toggles, and copy buttons.

```
[shared]
partials = ["site-meta.html", "theme-init.html", "styles.html", "scripts.html",
```

```
"pagefind-modal.html", "theme-toggle.html"]
styles = ["theme.css", "code.css", "widgets.css"]
scripts = ["theme-toggle.js", "language-picker.js", "copy-code.js"]
```

Shared imports load in the order listed in `theme.toml`. Files in the theme's own `partials/`, `styles/`, and `scripts/` directories load after the shared imports in filename order. If a theme-local file has the same filename as a shared import, the local file overrides that import.

Import names are filenames, not paths: use `theme.css`, not `styles/theme.css` or `../theme.css`. For local directory themes, *Calepin* first checks the theme's own file, then `themes/shared/`, then the embedded shared library shipped with *Calepin*. That means a new local theme can opt into shared files with only `theme.toml`, while an ejected theme also gives you editable copies under `themes/shared/`.

When you run `calepin new theme`, *Calepin* writes the selected theme into `themes/<name>/` and writes the shared library beside it in `themes/shared/` so you can inspect or edit the source files. Multiple ejected themes can share the same `themes/shared/` directory.

Shared partials

Shared partials live in `shared/partials/`:

```
themes/
  shared/
    partials/site-meta.html
    partials/theme-init.html
    partials/styles.html
    partials/scripts.html
    partials/pagefind-modal.html
    partials/theme-toggle.html
```

Themes include imported partials like normal MiniJinja partials:

```
{% include "partials/site-meta.html" %}
{% include "partials/styles.html" %}
```

The built-in themes share these partials for page metadata, early theme initialization, stylesheet output, script output, the Pagefind modal, and the generic theme-toggle button. Keep those imports when you want the same behavior, or remove individual names from `theme.toml` to take full control in your theme.

Shared CSS

Shared CSS lives in `shared/styles/`:

```
themes/
  shared/
    styles/theme.css
    styles/code.css
    styles/widgets.css
```

Put broad variables and base rules first in `theme.toml`, then component rules, then project-specific files in your theme:

```
themes/my-theme/
  theme.toml
  styles/90-overrides.css
```

If `styles/widgets.css` exists in your theme and `widgets.css` is also listed in `[shared].styles`, the theme-local file replaces the shared one. If `styles/90-overrides.css` has a different filename, it loads after all shared styles.

`theme.css` is the shared visual base used by the bundled themes. It defines common typography, heading sizes, accent variables, Pico primary colors, code/output variables, figure defaults, and global document defaults. Theme-specific CSS should generally be limited to the HTML shell and layout differences that cannot be shared.

`widgets.css` pairs with the shared JavaScript files.

Shared JavaScript

Shared JavaScript lives in `shared/scripts/`:

```
themes/
  shared/
    scripts/theme-toggle.js
    scripts/language-picker.js
    scripts/copy-code.js
```

Keep shared behavior before project-specific behavior by listing shared scripts first and putting custom scripts in your theme:

```
themes/my-theme/
  theme.toml
  scripts/90-custom.js
```

As with styles, a same-named local script replaces a shared script; a differently named local script loads after the shared scripts.

The shared JavaScript files expect these attributes:

```
<button data-calepin-theme-toggle>Theme</button>
<select data-calepin-language-picker></select>
<select id="calepin-website-view-mode"></select>
```

- `theme-toggle.js` enhances buttons marked with `data-calepin-theme-toggle`
- `language-picker.js` enhances selects marked with `data-calepin-language-picker`
- The website view switcher expects a select with `id="calepin-website-view-mode"`

Notebook Typst templates

Notebook outputs use a Typst-source MiniJinja template named `notebook.typ.jinja`:

```
themes/my-theme/
  notebook.typ.jinja
```

Typst already has a complete styling system based on `#set` and `#show` rules. *Calepin*'s notebook theme layer adds one optional step before Typst runs: `notebook.typ.jinja` is rendered with MiniJinja and must produce Typst source. This keeps the customization model similar to HTML themes, exposes *Calepin*-specific values, and still lets Typst handle the actual notebook rendering.

Every bundled theme includes `notebook.typ.jinja`. The selected theme's template is enabled by default for `calepin compile`, `calepin watch`, and website page builds. To customize it, eject a bundle:

```
calepin new theme
```

Then edit `themes/calepin/notebook.typ.jinja` and select that local theme:

```
calepin compile paper.typ --theme themes/calepin
```

In a website, use the same local theme directory in `calepin.toml`:

```
theme = "themes/calepin"
```

Internally, *Calepin* uses the notebook template while preparing the Typst file that Typst will compile:

1. *Calepin* preprocesses the notebook and writes a staged Typst source file under `.calepin/`.
2. *Calepin* renders `notebook.typ.jinja` with MiniJinja.
3. `document.body` expands to a Typst `#include` for the staged notebook source.
4. *Calepin* writes a wrapper file that contains the rendered theme and any notebook execution rules.
5. Typst compiles that wrapper to the requested output format.

For example, this template:

```
#set page(numbering: "1")

{{ document.body }}

[#align(center)[Generated by Calepin]]
```

becomes Typst source like this:

```
#set page(numbering: "1")

#include "/.calepin/paper/source.typ"

[#align(center)[Generated by Calepin]]
```

The exact `.calepin/.../source.typ` path is generated by *Calepin*. You normally do not write that path yourself; use `{{ document.body }}`.

The built-in notebook template currently imports its fenced-code helper from the runtime module: `/.calepin/calepin.typ`. That runtime is assembled from `src/assets/typst-runtime/*.typ` and includes `code-block` and the notebook rendering helpers. HTML-only syntax themes are generated by Rust during preprocessing so browser output can map Typst's compiled colors to CSS classes.

The template can be mostly plain Typst when no *Calepin* variables are needed. Use `document.body` where the notebook source should appear:

```
#set page(
  paper: "us-letter",
  margin: (x: 1in, y: 0.85in),
  numbering: "1",
)

#set text(font: "Libertinus Serif", size: 10.5pt)
#set heading(numbering: "1.1")

#show heading.where(level: 1): it => {
  pagebreak(weak: true)
  text(size: 18pt, weight: "semibold", it)
}

{{ document.body }}
```

Because the rendered output is Typst, it can also import project files such as `#import "/styles/report.typ": *`. Root-relative imports start from the website or document root.

`notebook.typ.jinja` receives:

- `theme`: the local theme directory name
- `target`: `notebook`
- `document.path`: the root-relative `.typ` input path
- `document.dir`: the root-relative input directory
- `document.stem`: the input filename without `.typ`
- `document.body`: the staged notebook source as a Typst `#include`
- `document.meta`: metadata from `#metadata(...) <website-metadata>`
- `params`: document parameters after CLI overrides

Use those values to insert generated front matter, appendices, disclaimers, or labels from *Calepin* metadata:

```
{% if document.meta.title %}
#set page(footer: align(right)[{{ document.meta.title }}])
{% endif %}

{{ document.body }}

{% if document.meta.appendix %}
pagebreak(weak: true)
heading("Appendix")
{{ document.meta.appendix }}
{% endif %}
```

If `notebook.typ.jinja` does not reference `document.body`, *Calepin* treats it like a prelude and includes the notebook source after the rendered template.

For output-specific branches, use Typst's runtime input instead of MiniJinja:

```
#let is-html = sys.inputs.at("calepin-target", default: "paged") == "html"
```

Set `theme = false` or use an empty `notebook.typ.jinja` to disable notebook Typst styling. Local themes that still use the older `paged.typ.jinja` filename continue to work, but new themes should use `notebook.typ.jinja`.